

Visual Studio Code for Image Analysis

Installation, extensions, interface, Python, notebooks, and Pixi environments

NanoCell Interactions Lab

2026-07-27

Table of contents

1	1. Install VS Code	3
1.1	1.1. Enable the code command	3
2	2. Install the extensions used in the course	4
2.1	2.1. Essential extensions	4
2.2	2.2. Auxiliary extensions	5
3	3. Explore the interface	6
3.1	3.1. Hands-on — Exploring the interface	7
4	4. Use the Command Palette	7
4.1	4.1. Hands-on — Editing commands	8
5	5. Create and run a Python file	9
5.1	5.1. Select the Python interpreter	9
5.2	5.2. Run the file	10
6	6. Work with a Pixi project	10
6.1	6.1. Install the environment	11
6.2	6.2. Open the project	11
6.3	6.3. Identify the environment in VS Code	11
7	7. Work with Jupyter notebooks	12
7.1	7.1. Test the packages	12
8	8. Understand the integrated terminal	13

8.1	8.1. Use <code>pixi shell</code>	14
9	9. Formatting, static analysis, and code organization	15
9.1	9.1. Black Formatter	15
9.2	9.2. isort	16
9.3	9.3. Pylint and static analysis	16
10	10. Configure line-length rulers	17
11	11. Integrated challenge	18
11.1	11.1 Part 1 — Create the script	18
11.2	11.2 Part 2 — Explore VS Code	19
11.3	11.3 Part 3 — Create a notebook	19
12	12. Checklist before starting an analysis	19
13	13. Common problems	20
14	14. Main messages	21
15	15. Further reading	21

Visual Studio Code, or VS Code, will be one of the tools used most frequently throughout this course. It allows you to work in the same application with:

- text and Markdown files;
- Python scripts;
- Jupyter notebooks;
- configuration files such as `pixi.toml`;
- tables and other project files;
- a terminal;
- version control with Git.

VS Code is an extensible editor. This means that new functions can be added through extensions, also called *plugins*. With the appropriate extensions, it becomes an integrated environment for editing, running, reviewing, and organizing computational analyses.

i Is VS Code an IDE?

VS Code is frequently used as an integrated development environment, or IDE, although its basic installation is closer to a code editor. Extensions add features such as Python execution, notebooks, static analysis, formatting, debugging, and integration with computational environments.

! Objective of this tutorial

The objective is not to explore every VS Code feature. By the end of the tutorial, you should be able to:

1. open a project folder;
2. locate and edit files;
3. install the extensions used in the course;
4. use the Command Palette;
5. open the integrated terminal;
6. select a Python interpreter;
7. select a notebook kernel;
8. work with a Pixi environment;
9. create and run a simple Python file.

1 1. Install VS Code

Download the installer from the official website:

Download Visual Studio Code

Choose the version for your operating system and follow the installation instructions.

After installation, open the application to confirm that it starts normally.

1.1 1.1. Enable the code command

The command:

```
code .
```

opens the current terminal folder in VS Code. The dot (.) means the current folder.

1.1.1 macOS

1. open VS Code;
2. open the Command Palette with `⇧⌘P`;
3. search for:

```
Shell Command: Install 'code' command in PATH
```

4. run the command;
5. close and reopen the terminal.

1.1.2 Windows

The installer normally adds the code command to the PATH. If it does not work, close and reopen the terminal. If necessary, run the installer again and check the option to add VS Code to the PATH.

1.1.3 Test

In the terminal, run:

```
code --version
```

If a version number is displayed, the command is available.

2 2. Install the extensions used in the course

Open the Extensions panel using:

- Windows/Linux: Ctrl+Shift+X;
- macOS: ⌘+X;

or go to:

```
View > Extensions
```

Search for each extension by name and also check the publisher before installing it.

2.1 2.1. Essential extensions

Install these extensions first:

Extension	Publisher	Main purpose
Python	Microsoft	Python language support, execution, debugging, and interpreter selection
Pylance	Microsoft	Autocompletion, code navigation, type analysis, and assistance while writing
Jupyter	Microsoft	Opening and running .ipynb notebooks
Pixi Code	Renan R. Santos	Integration of Pixi environments with VS Code
Black Formatter	Microsoft	Automatic and consistent formatting of Python files
isort	Microsoft	Automatic organization of import statements
Pylint	Microsoft	Static analysis and identification of potential problems in code

Automatic installation of dependencies

Some extensions may suggest or install related extensions. For example, the Python extension may recommend Pylance. Before installing it again, check whether the extension already appears as installed.

2.2 2.2. Auxiliary extensions

These extensions are not required to complete the course, but they may make writing easier:

Extension	Publisher	Purpose
Python Snippets	Ferhat Yalçın	Shortcuts for inserting common Python structures
autoDocstring — Python Docstring Generator	Nils Werner	Generates documentation templates for functions
Python Indent	Kevin Rose	Helps with indentation in Python structures

Essential and optional extensions

Begin with the essential extensions. Installing many extensions at once can make the interface more complex and make it harder to identify the source of warnings or unexpected behavior.

3 3. Explore the interface

VS Code organizes work around a project folder, also called a *workspace*. Whenever possible, open the entire project folder rather than an isolated file.

The left sidebar is called the Activity Bar. The main panels we will use are:

1. Explorer — displays the project files and folders;
2. Search — searches for words or text across all files;
3. Source Control — displays changes tracked by Git;
4. Run and Debug — runs and debugs code;
5. Extensions — installs and manages extensions;
6. Testing — runs tests when the project includes a testing structure.

At the bottom of the window is the Panel, which may contain:

- terminal;
- detected problems;
- extension output;
- debug console.

The bar at the bottom is called the Status Bar. Depending on the open file, it may display:

- selected Python interpreter;
- Git branch;
- file encoding;
- line-ending type;
- file language;
- environment-related messages.

3.1 Hands-on — Exploring the interface

1. Create a folder called `vscode-tutorial`.
2. Open VS Code.
3. Go to File > Open Folder.
4. Select the folder you created.
5. In Explorer, create:
 - a folder called `data`;
 - a file called `README.md`;
 - a file called `welcome.py`.
6. Use the Search panel to search for the word `welcome`.
7. Open and close the integrated terminal.

The structure should look like this:

```
vscode-tutorial/  
├─ data/  
├─ README.md  
└─ welcome.py
```

Confirm the open folder

The project folder name appears in Explorer. Before creating files or running commands, confirm that VS Code is open in the correct folder.

4 4. Use the Command Palette

The Command Palette allows you to locate functions without memorizing which menu contains them.

Open the palette using:

- Windows/Linux: `Ctrl+Shift+P`;
- macOS: `⌘P`;
- or press `F1`.

4.1 Hands-on — Editing commands

In `README.md`, write:

```
microscopy
python
image
python
pixi
```

Now try the following:

4.1.1 4.1. Transform text to uppercase

1. select a word;
2. open the Command Palette;
3. search for:

```
Transform to Uppercase
```

4.1.2 4.2. Sort lines

1. select all lines;
2. open the Command Palette;
3. search for:

```
Sort Lines Descending
```

4.1.3 4.3. Create multiple cursors

Add cursors to several lines using:

- Windows/Linux: `Alt + click`;
- macOS: `Option (⌘) + click`.

Type the following at the beginning of the lines:

```
item =
```

Multiple cursors allow you to repeat the same edit in several locations.

4.1.4 4.4. Select matching occurrences

Place the cursor on one occurrence of `python` and use:

- Windows/Linux: `Ctrl+D`;
- macOS: `⌘D`.

Press the shortcut again to select the next occurrence. Then edit the selected words at the same time.

i Not all shortcuts are identical across systems

Shortcuts may vary depending on the operating system, keyboard, and local customizations. The Command Palette is a reliable alternative when a shortcut does not work.

5 5. Create and run a Python file

Open `welcome.py` and write:

```
name = "HCI/HCA"

print(f"Welcome to the {name} course!")
```

Save the file using:

- Windows/Linux: `Ctrl+S`;
- macOS: `⌘S`.

5.1 5.1. Select the Python interpreter

Open the Command Palette and search for:

Python: Select Interpreter

Choose the interpreter associated with the project environment.

The selected interpreter is used by features such as:

- running scripts;
- debugging;
- autocompletion;
- code analysis;
- formatting and linting, depending on the configuration.

5.2 5.2. Run the file

With the file open, you can use the run button in the upper-right corner or open the terminal and run:

```
python welcome.py
```

When working with Pixi, the most explicit method is:

```
pixi run python welcome.py
```

The output should be:

```
Welcome to the HCI/HCA course!
```

6 6. Work with a Pixi project

In a Pixi project, the main folder usually contains:

```
project/  
├─ pixi.toml  
├─ pixi.lock
```

```
| notebooks/  
| data/  
| scripts/
```

The `pixi.toml` file describes the environment and its dependencies. The `pixi.lock` file records the resolved versions to make execution more reproducible.

6.1 6.1. Install the environment

In the terminal, enter the folder that contains `pixi.toml` and run:

```
pixi install
```

If `pixi.toml` already exists, you do not need to run `pixi init`.

6.2 6.2. Open the project

In the same folder, run:

```
code .
```

You do not need to activate the environment with `pixi shell` before opening VS Code.

6.3 6.3. Identify the environment in VS Code

With the Python and Pixi Code extensions installed:

1. open the Command Palette;
2. run Python: Select Interpreter;
3. look for the Python interpreter associated with the Pixi project;
4. select it.

The path is usually inside the `.pixi` folder, although its exact appearance may vary.

! Opening with `code .` does not automatically select the kernel

Opening VS Code from a terminal in which the environment is active may help some tools find executables, but it does not guarantee that a notebook will use the correct environment.

Always check explicitly:

- the interpreter for `.py` files;
- the kernel for `.ipynb` notebooks.

7 7. Work with Jupyter notebooks

Open an `.ipynb` file or create a new notebook through the Command Palette:

```
Create: New Jupyter Notebook
```

In the upper-right corner of the notebook, click:

```
Select Kernel
```

Choose the Python environment corresponding to the Pixi project.

Run a test cell:

```
import sys

print(sys.executable)
print(sys.version)
```

The path displayed by `sys.executable` should correspond to the intended environment.

7.1 7.1. Test the packages

Run:

```
import matplotlib
import numpy
import pandas

print("Environment ready.")
```

If the cell runs without errors, these packages are available in the selected kernel.

 Interpreter and kernel are not exactly the same selection

The interpreter selected for Python files and the kernel selected for notebooks may appear in different lists. A script may be configured for one environment while the notebook runs in another.

Check the kernel in the upper-right corner of each notebook.

8 8. Understand the integrated terminal

Open the terminal using:

```
Terminal > New Terminal
```

or use:

- Windows/Linux: Ctrl+`;
- macOS: Control+`.

The integrated terminal works like a normal terminal, but it is displayed inside VS Code.

 The terminal is not the same as the interpreter or kernel

Selecting an environment in VS Code does not guarantee that every command typed in an already-open terminal will use that environment.

Before installing packages or running an analysis, confirm where the command will be executed.

To check which Python is available in the terminal:

8.0.1 macOS/Linux/Git Bash

```
which python  
python --version
```

8.0.2 Windows PowerShell or Command Prompt

```
where.exe python  
python --version
```

With Pixi, prefer to run commands explicitly:

```
pixi run python --version  
pixi run python script.py  
pixi run notebook
```

This avoids depending on manual environment activation.

8.1 8.1. Use `pixi shell`

To activate the environment in the terminal:

```
pixi shell
```

Then check:

```
python --version
```

To leave the environment:

```
exit
```

 When should you use each approach?

Use `pixi run` when you want to run a task explicitly and reproducibly:

```
pixi run python script.py
```

Use `pixi shell` when you want to run several consecutive commands inside the environment:

```
pixi shell
python script_1.py
python script_2.py
exit
```

9 9. Formatting, static analysis, and code organization

9.1 9.1. Black Formatter

Black automatically organizes the style of Python code.

To format a file:

1. open the Command Palette;
2. search for:

```
Format Document
```

3. if prompted, choose Black Formatter.

To make Black the default formatter:

1. open a `.py` file;
2. open the Command Palette;
3. run:

```
Format Document With...
```

4. select:

```
Configure Default Formatter
```

5. choose Black Formatter.

9.2 9.2. isort

isort organizes import statements.

Example before organization:

```
import pandas as pd
from pathlib import Path
import numpy as np
```

After organization:

```
from pathlib import Path

import numpy as np
import pandas as pd
```

Use the Command Palette and search for:

```
Organize Imports
```

9.3 9.3. Pylint and static analysis

A static analyzer examines code without needing to execute all of its instructions. It can identify:

- undefined names;
- unused imports;
- unused variables;
- style problems;

- potentially problematic constructs.

Pylint may underline code and send messages to the panel:

View > Problems

i A warning does not necessarily mean an execution error

A line highlighted by Pylint may still run. Read and evaluate the message: some warnings indicate real errors, while others are recommendations about style or maintainability.

10 10. Configure line-length rulers

Vertical rulers help you notice when a line has become too long.

Open Settings using:

- Windows/Linux: Ctrl+,;
- macOS: ⌘,.

Search for:

editor.rulers

Or open the settings as JSON:

Preferences: Open User Settings (JSON)

Add:

```
"editor.rulers": [80, 120]
```

This creates rulers at columns 80 and 120.

To adjust the color:

```
"workbench.colorCustomizations": {  
    "editorRuler.foreground": "#ff4081"  
}
```

💡 User setting or project setting?

- User Settings affect every project opened by the user.
- Workspace Settings affect only the current project and can be saved in `.vscode/settings.json`.

Settings specific to the course or pipeline can be stored in the workspace to make behavior more consistent across participants.

11 11. Integrated challenge

Use the `vscode-tutorial` project created earlier.

11.1 Part 1 — Create the script

Replace the contents of `welcome.py` with:

```
from pathlib import Path  
  
course = "HCI/HCA"  
files = ["image.tif", "measurements.csv", "notebook.ipynb"]  
  
folder = Path("data")  
  
print(f"Course: {course}")  
print(f>Data folder: {folder.resolve()}")  
  
for file in files:  
    print(f"- {file}")
```

11.2 Part 2 — Explore VS Code

1. select the Python interpreter;
2. format the document with Black;
3. organize the imports;
4. open the Problems panel;
5. run using:

```
pixi run python welcome.py
```

if the project has a Pixi environment; or:

```
python welcome.py
```

if you are using another configured environment.

11.3 Part 3 — Create a notebook

Create `test_environment.ipynb` and run:

```
import sys
from pathlib import Path

print(sys.executable)
print(Path.cwd())
```

Compare:

- the kernel path;
- the working directory;
- the path shown by the terminal.

12 12. Checklist before starting an analysis

Before running a notebook or pipeline, confirm:

- ☐ I opened the correct project folder;
- ☐ `pixi.toml` is in the expected folder;
- ☐ I ran `pixi install`;
- ☐ I selected the correct interpreter;
- ☐ I selected the correct notebook kernel;
- ☐ I checked the path displayed by `sys.executable`;
- ☐ I know which folder the terminal is currently using;
- ☐ I am using `pixi run` or a Pixi shell when necessary;
- ☐ I am not accidentally installing packages in the base environment or global Python.

13 13. Common problems

Problem	Possible cause	What to check
code: command not found	The code command is not in the PATH	Install the command through the Command Palette on macOS or reopen/reinstall on Windows
Python does not appear	The environment has not been installed or detected	Run <code>pixi install</code> , reload the window, and select the interpreter
The notebook keeps loading indefinitely	Incorrect kernel, missing <code>ipykernel</code> , or inconsistent environment	Check Select Kernel and test <code>pixi run python -c "import ipykernel"</code>
The script works, but the notebook does not	Interpreter and kernel are different	Check the kernel in the upper-right corner
A package cannot be found	The code was executed in another environment	Check <code>sys.executable</code> and use <code>pixi run</code>
The terminal displays (base)	Conda automatically activated the base environment	Do not use <code>conda install</code> ; run through Pixi or open a new terminal
The extension cannot find the environment	The open folder is not the project root	Open the folder that contains <code>pixi.toml</code>
VS Code displays many warnings	A linter, type analyzer, or other extensions are active	Read the Problems panel and distinguish errors from recommendations

To reload VS Code without closing the project:

1. open the Command Palette;
2. run:

```
Developer: Reload Window
```

14 14. Main messages

VS Code organizes files, code, notebooks, extensions, and a terminal in a single interface.

Opening the project with code . does not replace selecting the interpreter and kernel.

The interpreter runs Python scripts; the kernel runs notebook cells; the terminal runs the commands typed into it.

In Pixi projects, `pixi run` is the most explicit way to ensure that a command runs in the correct environment.

Do not rely only on the displayed environment name. Check the path using `sys.executable`, which `python`, or `where.exe python`.

15 Further reading

- [Official Visual Studio Code documentation](#)
- [Python in Visual Studio Code](#)
- [Python environments in Visual Studio Code](#)
- [Jupyter notebooks in Visual Studio Code](#)
- [Jupyter kernel management](#)
- [VS Code command-line interface](#)
- [Pixi Code on the Visual Studio Marketplace](#)